

```

# =====
# Student Name : Srushti Kale
# PRN          : 23070521067
# Experiment 6 : Handwritten Digit Recognition using CNN
# =====

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# --- 1. Load and Preprocess MNIST Dataset ---
# Srushti Kale - 23070521067: Loading the MNIST handwritten digits dataset
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()

# Normalize pixel values from [0, 255] to [0.0, 1.0]
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Reshape images to include channel dimension (28, 28, 1) for Conv2D
X_train = np.expand_dims(X_train, -1)
X_test = np.expand_dims(X_test, -1)

print(f"Training shape: {X_train.shape}")
print(f"Testing shape : {X_test.shape}")

# --- 2. Build CNN Architecture ---
# Srushti Kale - 23070521067: Defining Convolutional, Pooling, and Dense layers
model = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),

    # First Conv Block
    layers.Conv2D(32, kernel_size=(3, 3), activation="relu"),
    layers.MaxPooling2D(pool_size=(2, 2)),

    # Second Conv Block
    layers.Conv2D(64, kernel_size=(3, 3), activation="relu"),
    layers.MaxPooling2D(pool_size=(2, 2)),

    # Classification Head
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax") # 10 output classes for digits 0-9
])

model.summary()

# --- 3. Model Compilation ---
# Srushti Kale - 23070521067: Using Adam optimizer and Sparse Categorical Crossentropy loss
model.compile(
    optimizer="adam",

```

```

        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )

# --- 4. Train the CNN Model ---
EPOCHS = 5
BATCH_SIZE = 128

history = model.fit(
    X_train, y_train,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    validation_split=0.1,
    verbose=1
)

# --- 5. Evaluate Model ---
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\n--- Evaluation Results (Srushti Kale - 23070521067) ---")
print(f"Test Loss      : {test_loss:.4f}")
print(f"Test Accuracy: {test_acc * 100:.2f}%")

# --- 6. Display Prediction Results ---
# Srushti Kale - 23070521067: Generating predictions and plotting sample results
predictions = model.predict(X_test[:10])
predicted_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(12, 5))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap="gray")
    plt.title(f"Pred: {predicted_labels[i]} | True: {y_test[i]}")
    plt.axis("off")

plt.suptitle("Sample Predictions - Srushti Kale (23070521067)", fontsize=14)
plt.tight_layout()
plt.show()

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-data/11490434/11490434> 0s 0us/step

Training shape: (60000, 28, 28, 1)

Testing shape : (10000, 28, 28, 1)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dropout (Dropout)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

422/422 46s 105ms/step - accuracy: 0.9181 - loss: 0.2664

Epoch 2/5

422/422 43s 103ms/step - accuracy: 0.9735 - loss: 0.0855

Epoch 3/5

422/422 43s 101ms/step - accuracy: 0.9790 - loss: 0.0650

Epoch 4/5

422/422 42s 101ms/step - accuracy: 0.9833 - loss: 0.0536

Epoch 5/5

422/422 83s 103ms/step - accuracy: 0.9856 - loss: 0.0440

--- Evaluation Results (Srushti Kale - 23070521067) ---

Test Loss : 0.0271

Test Accuracy: 99.05%

1/1 0s 105ms/step

Sample Predictions - Srushti Kale (23070521067)



